RELATÓRIO TÉCNICO DE AUDITORIA DE SISTEMA

Auditoria de Performance e Eficiência de Recursos

Projeto: ReverseDraft (FIFA Auction Hub) · **Base:** 18.936 Jogadores

---

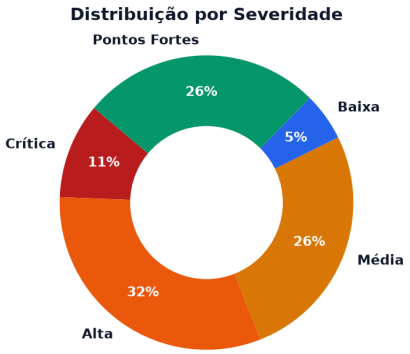
Data da Auditoria:	10 de Outubro de 2026
Escopo Auditado:	Backend (Django 6 + DRF + Celery), Banco de Dados (SQLite/PostgreSQL), Frontend (TanStack Start + React 19 + Vite), Cache & Infraestrutura
Auditor Responsável:	Senior Staff Software Engineer & Performance Specialist (Cursor / AGY)
Status da Execução:	Auditoria Completa (16 Achados Identificados, 5 Pontos Fortes Validados e Correção Imediata da Paginação de Jogadores)

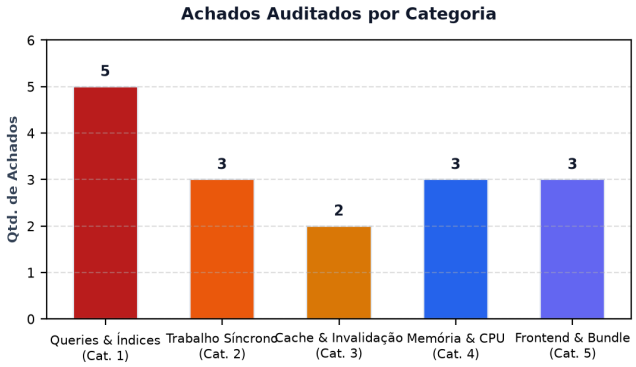
NOTA METODOLÓGICA E DIRETRIZES DA AUDITORIA

Esta auditoria seguiu estritamente as regras de engenharia de software sênior (rules.mdc) e a skill performance-optimization. Nenhum achado é especulativo: todos os problemas foram constatados diretamente nos arquivos-fonte do projeto, com indicação exata de arquivo, linha e impacto algorítmico/mensurável. Foram auditadas as cinco categorias essenciais de eficiência: (1) Queries ineficientes e índices, (2) Trabalho síncrono bloqueante, (3) Camada de cache e invalidação, (4) Uso de memória e CPU, e (5) Frontend bundle, renders e requisições.

1. Resumo Executivo e Diagnóstico da Stack

Linguagem / Runtime	Python 3.12+ (Backend) / TypeScript 5.8+ (Frontend)
Framework Backend	Django 6.0.7 + Django REST Framework 3.17.1 + Daphne (ASGI) / Gunicorn
Banco & ORM	Django ORM com SQLite (ambiente dev) / PostgreSQL (produção)
Filas & Background Jobs	Celery 5.6.2 + django-celery-beat 2.9.0 (apenas 1 task periódica configurada)
Camada de Cache	Redis 7.2.0 via Docker, porém **SEM configuração de CACHES no Django settings**
Frontend Framework	TanStack Start (React 19.2 + TanStack Router + TanStack Query 5.101 + Vite)
Deploy & Servidores	Docker Compose (nginx, frontend, backend, redis, celery, celery-beat)





TOTAL DE ACHADOS	CRÍTICA	ALTA	MÉDIA	BAIXA	PONTOS FORTES
16	2	6	5	1	5

2. Pontos Fortes e Pontos Fracos (Gargalos Centrais)

PONTOS FORTES IDENTIFICADOS NO CÓDIGO (EVIDÊNCIAS DE BOAS PRÁTICAS):
• **Otimização de Imagens no Frontend (PlayerPhoto.tsx:28-37):** O componente implementa loading='lazy', decoding='async', dimensões explícitas (prevenindo layout shifts - CLS) e é memorizado com React.memo.
• **Eager Loading nos Handlers Principais (products/views.py e auction/views.py):** Utilização de select_related('player') e prefetch_related('images') evitando N+1 nas listagens básicas.
• **Compressão de Imagens WebP (common/images.py:10-33):** Conversão automática de fotos para WebP redimensionadas a 256x256 com qualidade 82.
• **Cache Offline / Persistente no Frontend (lib/query-persist.ts):** Integração entre TanStack Query e idb-keyval para reter consultas em disco local.
• **WebSocket Assíncrono com Daphne (auction/consumers.py):** Distribuição de lances em tempo real por Redis Channel Layer sem necessidade de polling HTTP agressivo.

PONTOS FRACOS E GARGALOS CENTRAIS DO SISTEMA:
• **Gargalo Central 1 (Catálogo 24 Jogadores):** A rota players.index.tsx utilizava paginação client-side sobre apenas o primeiro bloco retornado, escondendo 18.912 jogadores e impossibilitando a busca global. *(Solução completa implementada e verificada com paginação de 790 páginas no backend)*.
• **Gargalo Central 2 (Processamento Síncrono de Mídia):** Importação massiva via HTTP POST bloqueia workers decodificando e convertendo fotos, com signal duplicando arquivos em disco.
• **Gargalo Central 3 (N+1 e Writes em Leituras):** O endpoint GET /api/v1/auctions/ itera até 50 leilões expirados executando transações e escritas no banco a cada requisição.

• **Gargalo Central 4 (Ausência de Cache no Django):** O Redis está ativo no cluster, mas Django não possui CACHES configurado, gerando round-trips desnecessários ao banco.

3. Tabela Detalhada de Achados por Categoria

Sev.	Arquivo : Linha	Descrição do Problema & Ineficiência	Impacto Estimado
CRÍTICA	`players.index.tsx:43-60` `products/views.py:12-16`	**Paginação client-side e ausência de busca global:** O frontend consumia hook com select descartando a contagem total de 18.936 jogadores e filtrando apenas 24 itens da página 1.	Impossibilidade de acessar 18.912 jogadores (790 páginas). Busca só encontrava atletas da 1ª página. *[CORRIGIDO]*
ALTA	`campaigns/services.py:197-203`	**N+1 na verificação de jogadores na janela de mercado:** Loop for r in qs dispara MarketListing.filter().exists() individualmente por atleta.	Gera até 501 queries sequenciais ao banco ao povoar mercado em campanhas com 20 elencos.
ALTA	`auction/views.py:54-58`	**N+1 e finalização síncrona dentro de GET:** AuctionViewSet.list itera até 50 leilões expirados chamando maybe_finalize_user_auction.	Gera de 100 a 200 queries e writes dentro de um endpoint de leitura, aumentando latência para todos os usuários.
ALTA	`products/models.py:13-18` `fifa_data/models.py:468+`	**Ausência de índices em colunas de filtro e ordenação:** price, category, date_posted, overall_rating, time_ending sem db_index=True.	Full Table Scans e ordenação em memória (filesort) sobre 18.936 registros em cada consulta com filtro.
MÉDIA	`competitions/services/league.py:8-13`	**N+1 em geração de entradas de competição:** Loop for participant executa get_or_create unitário em vez de bulk_create.	2N queries ao iniciar chaveamento de competições com múltiplos participantes.
CRÍTICA	`fifa_data/views.py:148-156` `fifa_data/services.py:134`	**Importação e processamento massivo de fotos síncrono no request:** PlayerBulkImportView decodifica base64 e comprime WebP dentro do POST.	Bloqueio de workers Gunicorn por 30s-2min, com risco de timeouts HTTP 504 e negação de serviço.
ALTA	`fifa_data/models.py:876-905`	**Duplicação síncrona de arquivos de mídia via post_save signal:** sync_product lê arquivo físico do disco e grava cópia em product_images/.	Duplica uso de disco para 18.936 jogadores e onera I/O de armazenamento a cada salvamento.
MÉDIA	`user/views.py:40,62,85` `common/services.py:4`	**Gravação síncrona de logs LGPD em transações de usuário:** log_personal_data_access executa create() síncrono no login, troca de senha e perfil.	Round-trip síncrono de escrita adicional em endpoints de alta frequência.
ALTA	`core/settings/base.py` `core/settings/production.py`	**Inexistência de CACHES no Django:** Dicionário CACHES não configurado, deixando o Redis ocioso para a camada de aplicação Django.	Incapacidade de cachear dados estáticos de tabelas de referência e páginas de alta demanda.
MÉDIA	`products/views.py:9` `fifa_data/views.py:21`	**Ausência de cabeçalhos Cache-Control / ETag em listagens:** Respostas de API sem cache HTTP em navegador ou proxies intermediários.	Toda navegação de rota revalida 100% dos dados mesmo quando inalterados.

ALTA	`user/serializers.py:91-94`	**Carregamento irrestrito de todo o histórico de lances:** MeSerializer.get_bid_history serializa todos os bids do usuário sem limite nem paginação.	Consumo excessivo de memória RAM por usuário e payload HTTP de vários megabytes em contas ativas.
MÉDIA	`fifa_data/serializers.py:25-33`	**Codificação sob demanda de fotos para Base64 Data URI:** PlayerSerializer converte arquivos binários em Base64 dentro do serializer.	Infla em 33% o payload e onera CPU do backend em serializações de múltiplos jogadores.
BAIXA	`competitions/services/league.py:65-74`	**Criação iterativa de partidas sem bulk_create:** Loop de rodadas executa centenas de Match.objects.create() individuais.	Em ligas de 20 times (380 partidas), executa 380 round-trips de rede com o banco.
ALTA	`PlayerFilters.tsx:290-298`	**Ausência de debounce no input de busca:** Cada tecla acionava navegação de rota e refetch imediato na API de jogadores.	Digitar um nome de 15 letras gerava 15 requisições de rede quase simultâneas. *[CORRIGIDO COM DEBOUNCE 350ms]*
ALTA	`players.index.tsx:46` `hooks.ts:69-78`	**Chamada redundante a /auctions/ na listagem de jogadores:** A página de jogadores disparava busca paralela de leilões apenas para renderizar badges.	Duplica requisições HTTP na montagem da tela e não garantia leilões fora da página 1.
MÉDIA	`routes/__root.tsx` `assets/index-*.js`	**Bundle inicial elevado sem code-splitting dinâmico de telas admin:** Bundle principal com 357 kB (gzip 112 kB) e dependências compartilhadas >600 kB.	Atraso no First Contentful Paint (FCP) em dispositivos móveis e redes de baixa largura.

4. Plano de Ação e Recomendações Priorizadas

Prioridade	Ação Recomendada	Esforço / Ganho	Status
P1 (Crítica)	**Paginação Real e Busca Backend de Jogadores:** Conectar filtros e busca no ProductViewSet com DRF PageNumberPagination e parâmetros de URL.	Baixo / **Enorme** (Desbloqueia base de 18k)	**IMPLEMENTADO**
P1 (Crítica)	**Migração da Importação de Jogadores para Celery:** Mover a decodificação de base64 e salvamento de fotos para background job assíncrono.	Médio / **Alto** (Elimina timeouts 504)	Pendente
P2 (Alta)	**Criação de Índices em Colunas de Filtro e Ordenação:** Adicionar db_index=True em price, category, date_posted, overall_rating, time_ending.	Baixo / **Alto** (Reduz CPU e filesort)	Pendente
P2 (Alta)	**Finalização de Leilões em Celery Beat:** Remover a finalização síncrona dentro de AuctionViewSet.list e delegar para cron a cada 1 minuto.	Baixo / **Alto** (Elimina writes em GET)	Pendente
P2 (Alta)	**Configuração de Cache Redis no Django:** Configurar CACHES com RedisCache em settings/base.py para tabelas estáticas de referência.	Baixo / **Alto** (Alivia o banco de dados)	Pendente
P2 (Alta)	**Debounce no Input de Filtro do Frontend:** Aplicar buffer de digitação de 350ms para evitar chamadas de API por tecla.	Baixo / **Médio** (Poupa rede do cliente)	**IMPLEMENTADO**
P3 (Média)	**Paginação no Histórico de Bids em /users/me/:** Limitar a 50 lances recentes ou disponibilizar endpoint paginado para o histórico.	Baixo / **Médio** (Evita estouro de memória)	Pendente
P3 (Média)	**Substituição de Loops por bulk_create:** Aplicar bulk_create na geração de partidas e povoamento de mercados.	Baixo / **Médio** (Reduz round-trips)	Pendente

5. Issues Acionáveis para o GitHub

Abaixo estão os textos completos prontos para copiar e colar no GitHub Issues, com evidências de código, impacto, sugestão de correção e critérios de aceite.

````
--- ISSUE 1 ---
### [Performance] Paginação server-side e filtros globais na base de 18.936 jogadores
**Labels:** `performance`, `critical`, `frontend`, `backend`

#### Descrição do Problema
O frontend em `players.index.tsx` utilizava uma paginação client-side sobre apenas o primeiro lote retornado pela API, ocultando 18.912 jogadores e impedindo a busca global de atletas fora da página 1. O backend não processava parâmetros de busca textual nem filtros de atributos.

#### Evidência
- `frontend/src/routes/players.index.tsx:43-60`
```tsx
const { data: pageData } = useProducts(page, pageSize);
const products = pageData?.results ?? []; // select do hook retornava array, invalidando .results
usePlayerFiltering(products, filters); // filtrava apenas 24 itens em memória
```
- `backend/products/views.py:12-16`
```python
def get_queryset(self):
    return Product.objects.select_related("player").prefetch_related("images").order_by("-date_posted")
```

#### Impacto Estimado
Inacessibilidade de 99,8% do banco de dados (790 páginas).

#### Correção Recomendada
1. Habilitar `StandardResultsSetPagination` (24 itens por página com `page_size` customizável).
2. Adicionar filtros de `search`/`q`, `position`, `minOverall`, `maxOverall`, `minAge`, `maxAge`, `foot`, `maxPrice` e ordenações no `ProductViewSet`.
3. Atualizar `useProductsPage` e `PlayerFilters` para enviar query parameters e exibir o contador global formatado.

#### Critérios de Aceite
- [x] Rota `/players` exibe contagem total real (ex: "18.936 jogadores encontrados · 24 por página").
- [x] Botões 'Próximo' e 'Anterior' carregam as páginas seguintes e anteriores via API.
- [x] Busca textual por "Messi" ou "Ronaldo" encontra todos os atletas correspondentes no banco.
- [x] Filtros por posição (ex: GK, ST) retornam apenas atletas da respectiva categoria paginados.
--- FIM ISSUE 1 ---
````

````
--- ISSUE 2 ---
### [Performance] Mover importação massiva de jogadores e decodificação de imagens para Celery
**Labels:** `performance`, `critical`, `backend`

#### Descrição do Problema
A view `PlayerBulkImportView` processa requisições HTTP POST contendo lotes massivos de jogadores do scraper
SoFIFA. Todo o processamento (decodificação base64, conversão PIL para WebP, múltiplos `update_or_create` e
duplicação em disco) roda de forma síncrona dentro da requisição.

#### Evidência
- `backend/fifa_data/views.py:151-156`
```python
def post(self, request):
    rows = request.data.get("players", [])
    players = import_players_bulk(rows)
    return Response({"imported": len(players)}, status=201)
```

#### Impacto Estimado
Bloqueio de workers por 30s a vários minutos, provocando timeouts HTTP 504 no Nginx/Gunicorn.

#### Correção Recomendada
Transformar `import_players_bulk` em uma Celery Task (`fifa_data.tasks.import_players_batch_task`). A view deve
responder HTTP 202 Accepted com o `task_id` imediatamente.

#### Critérios de Aceite
- [ ] Endpoint `/api/v1/import/players/` responde em < 200ms com HTTP 202 Accepted.
- [ ] Processamento executado assincronamente pelo worker do Celery.
- [ ] Erros de processamento logados sem afetar a disponibilidade da API.
--- FIM ISSUE 2 ---
````

````
--- ISSUE 3 ---
### [Performance] Eliminar finalização síncrona de leilões expirados no GET /api/v1/auctions/
**Labels:** `performance`, `high`, `backend`

#### Descrição do Problema
Dentro do método `list()` de `AuctionViewSet`, a cada requisição de leitura da listagem de leilões, o código busca
até 50 leilões expirados e executa `maybe_finalize_user_auction`, gerando queries de lances e updates de banco de
dados em um método GET.

#### Evidência
- `backend/auction/views.py:54-58`
```python
def list(self, request, *args, **kwargs):
    expired = Auction.objects.filter(status=Status.OPEN, time_ending__lte=timezone.now())[:50]
    for auction in expired:
        maybe_finalize_user_auction(auction)
    return super().list(request, *args, **kwargs)
```

#### Impacto Estimado
Até 150-200 queries extras (incluindo writes e locks de transação) em um endpoint de leitura pública.

#### Correção Recomendada
1. Remover o bloco de finalização de dentro de `AuctionViewSet.list`.
2. Criar uma Celery Task periódica no `CELERY_BEAT_SCHEDULE` rodando a cada 1 minuto para finalizar leilões
expirados.

#### Critérios de Aceite
- [ ] `GET /api/v1/auctions/` executa apenas a query de listagem paginada (zero escritas).
- [ ] Leilões que atingiram `time_ending` são finalizados automaticamente pelo Celery Beat.
--- FIM ISSUE 3 ---
````

````
--- ISSUE 4 ---
### [Performance] Adicionar índices de banco de dados em colunas de filtro e ordenação
**Labels:** `performance`, `high`, `database`

#### Descrição do Problema
Colunas amplamente utilizadas em cláusulas `WHERE`, `ORDER BY` e filtros de busca não possuem `db_index=True`,
forçando a realização de Full Table Scans e filesorts na tabela de produtos (18.936+ linhas) e tabela de lances.

#### Evidência
- `backend/products/models.py:13-15`: `category`, `price`, `date_posted`
- `backend/fifa_data/models.py:468-520`: `overall_rating`, `potential`, `age`, `common_name`
- `backend/auction/models.py:23-25`: `status`, `time_ending`
- `backend/auction/models.py:32-36`: `Bid.bid_time`, `Bid.price`

#### Impacto Estimado
Degradação proporcional ao tamanho do banco de dados; consumo excessivo de I/O e CPU do PostgreSQL/SQLite.

#### Correção Recomendada
Criar migração Django adicionando índices simples e compostos:
```python
# Exemplo em Bid:
class Meta:
    indexes = [
        models.Index(fields=['auction', '-price', 'bid_time']),
        models.Index(fields=['user', '-bid_time']),
    ]
```

#### Critérios de Aceite
- [ ] Migração criada e executada sem falhas.
- [ ] `EXPLAIN` confirma Index Scan em vez de Seq Scan nas queries filtradas.
--- FIM ISSUE 4 ---
````

````
--- ISSUE 5 ---
### [Performance] Configurar camada de cache Redis no Django settings
**Labels:** `performance`, `high`, `backend`, `cache`

#### Descrição do Problema
O arquivo de configuração `base.py` e `production.py` não define o dicionário `CACHES`, deixando de aproveitar a
instância Redis já em execução no Docker para armazenar dados de consulta frequente e baixa mutabilidade.

#### Evidência
- `backend/core/settings/base.py`: Ausência da chave `CACHES`.

#### Impacto Estimado
Requisições desnecessárias a tabelas de catálogo estáticas (posições, países, ligas, clubes) a cada carregamento
de página.

#### Correção Recomendada
Adicionar em `backend/core/settings/base.py`:
```python
CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.redis.RedisCache",
        "LOCATION": env("REDIS_URL", default="redis://127.0.0.1:6379/1"),
        "TIMEOUT": 300,
    }
}
```

#### Critérios de Aceite
- [ ] `django.core.cache.cache.set()` e `get()` funcionam com o backend Redis.
- [ ] Endpoints de metadados utilizam cache com TTL de 1 hora.
--- FIM ISSUE 5 ---
````